Face Enrollment Service

Production-Grade API & Pipeline Technical Guide

TECHNICAL MANUAL & SYSTEM ARCHITECTURE

Phase 1: Real-Time Quality Check & Candidate Enrollment

Document Version:	1.0.0 (Release)
AI Recognition Backend:	InsightFace buffalo_l (SCRFD + ArcFace w600k_r50)
Database Storage:	PostgreSQL (SQLAlchemy Async / Alembic Migrations)
Frontend Client:	Streamlit UI Application
Developed By:	Antigravity AI Agent Pair-Programmer
Date Generated:	July 8, 2026

1. Executive Summary

The Face Enrollment Service is a production-grade backend application engineered to handle candidate registration and facial biometric data extraction. It forms the foundational 'Phase 1' of a larger 1:N face recognition system. The core objective of this service is to ensure that only high-quality, validated facial images are enrolled into the system, and to generate L2-normalized 512-dimensional embeddings representing the facial characteristics of each candidate. By strictly checking quality parameters and utilizing state-of-the-art deep learning architectures, the system ensures a high level of accuracy and reliability for future downstream matching applications.

Key Architectural Principles

- Modularity: Clean separations between Presentation, Application, Domain, and Infrastructure layers.
- Robustness: Multi-tiered validation (HTTP, Image payload, and CV/ML quality constraints).
- Performance: Fully asynchronous database operations and highly optimized ONNX Runtime execution on CPU.
- Portability: Universal export interfaces (JSON & CSV) allowing seamless transfer to search indexes like FAISS.

Phase 1 Boundary Notice

This service implements Face Enrollment, Image Quality Verification, and Embedding Generation only. It does NOT perform face matching, cosine similarity searches, or FAISS index retrieval.

2. System Architecture

The application is structured into a clean, layered architecture to decouple concerns and ensure maintainability. Below is an overview of the structural layers:

1. Presentation Layer
   - FastAPI Routers: Serve REST endpoints for candidate profiles, webcam captures, and file imports.
   - SlowAPI Rate Limiter: Protects endpoints against brute-force calls based on IP addressing.
   - Logging Middleware: Custom interceptor capturing response status, latencies, and transaction IDs.
   - CORS Middleware: Secures access control policies for multi-origin client requests.

2. Application Layer
   - CandidateService: Handles candidate profile management and CRUD logic.
   - EnrollmentService: Coordinates image decoding, uniqueness checks, quality checks, and DB persistence.
   - EmbeddingService: Handles manual embedding retrieval and regeneration workflows.
   - Export/ImportServices: Executes database extracts and bulk data imports.

3. Domain Layer
   - Pydantic Schemas: Type-safe payload schemas enforcing strict runtime validation.
   - Custom Domain Exceptions: Translates infrastructure failures into user-friendly HTTP responses.

4. Infrastructure Layer
   - Repositories: SQLAlchemy async repositories for relational data operations.
   - FaceRecognitionPipeline: Encapsulates InsightFace, SCRFD, and ArcFace models.

- PostgreSQL Database: Storage for candidates and high-dimensional float vectors.

3. Machine Learning and Model Pipeline Workflow

The core of the application is the FaceRecognitionPipeline. It processes raw images in BGR format and coordinates four primary steps: Detection, Quality Verification, Alignment, and Embedding Generation.

Step-by-Step Pipeline Execution Flow

Step 1: Face Detection (SCRFD)
The system uses the Sample Consensus Robust Face Detector (SCRFD) loaded from the buffalo_l model pack. SCRFD is a high-performance face detector optimized for sub-millisecond execution. It locates face candidates in the input image and returns a bounding box (x1, y1, x2, y2), detection confidence, and 5 facial keypoints (left eye, right eye, nose tip, left mouth corner, right mouth corner).

Step 2: Quality Check Constraints
To ensure high-quality biometric data, the candidate face must pass a series of automated quality checks. If any check fails, the pipeline aborts immediately and returns a detailed failure code:
- Resolution Check: Image size must be at least 64x64 pixels.
- Face Count Check: Exactly one face must be detected (multiple faces or zero faces are rejected).
- Confidence Score: The face detector confidence score must be >= 0.50.
- Face Size Check: Bounding box area must be >= 4,000 px^2 (e.g., approx 63x63 px size inside the frame).
- Blur Detection: Laplacian variance of grayscale image must be >= 50.0 (lower values indicate blurriness).
- Brightness Range: Mean pixel brightness must be between 40.0 (too dark) and 220.0 (too bright).

Step 3: Face Alignment (norm_crop)
To ensure scale and rotation invariance, the face is normalized using an affine transform. The 5 facial keypoints obtained in Step 1 are mapped to target coordinates. The face is cropped and warped to a standard size of 112x112 pixels. This aligned image is fed to the embedding model.

Step 4: Embedding Extraction (ArcFace)
The aligned 112x112 RGB image is processed by the ArcFace w600k_r50 ResNet50 model. The neural network maps facial features to a highly discriminative 512-dimensional float vector. Finally, the embedding is L2-normalized so that its Euclidean norm is exactly 1.0. This allows subsequent matching systems to compute cosine similarity using a simple dot product.

4. Database Design and Entity Relationship

The system stores candidate profiles and their corresponding 512-d embeddings in a relational PostgreSQL database. Below is the relational schema definition:

Candidates Table Schema

Stores the demographic data of candidates. Each candidate must have a unique candidate_id and email.

```
  Table: candidates
-----------------------------------------------------------------
Column Name     | Type       | Constraints
-----------------------------------------------------------------
id              | UUID       | PRIMARY KEY, Default: uuid_generate_v4()
candidate_id    | VARCHAR    | UNIQUE, NOT NULL (e.g. Employee ID)
name            | VARCHAR    | NOT NULL
email           | VARCHAR    | UNIQUE, NOT NULL
dob             | DATE       | NULLABLE
created_at      | TIMESTAMP  | NOT NULL, Default: timezone('utc', now())
updated_at      | TIMESTAMP  | NOT NULL, Default: timezone('utc', now())
-----------------------------------------------------------------
```

Face Embeddings Table Schema

Stores the high-dimensional biometric vectors. Linked to the candidates table via a foreign key.

```
  Table: face_embeddings
-----------------------------------------------------------------
Column Name     | Type       | Constraints
-----------------------------------------------------------------
id              | UUID       | PRIMARY KEY, Default: uuid_generate_v4()
candidate_uuid  | UUID       | FOREIGN KEY -> candidates.id, ON DELETE CASCADE
embedding       | FLOAT[]    | NOT NULL (512 float array)
dimension       | INTEGER    | NOT NULL (default 512)
model_name      | VARCHAR    | NOT NULL (default 'buffalo_l')
normalized      | BOOLEAN    | NOT NULL (default TRUE)
created_at      | TIMESTAMP  | NOT NULL, Default: timezone('utc', now())
-----------------------------------------------------------------
```

5. Features and Client Applications

The system provides a rich set of operational capabilities accessible via REST API and a user-friendly UI.

Key Features

- Dual-Channel Enrollment: Register candidates via multipart file uploads or direct webcam capture.
- Automated Quality Inspector: Prevents low-quality, dark, blurry, or multi-face images from entering the DB.
- Paginated Search: Filter and list candidate profiles and retrieve matching ID records.
- Single and Bulk Export: Generate JSON or CSV format files containing candidate data and 512-d embeddings.
- Bulk Import: Read exported CSV or JSON sheets, skip duplicates, report detailed errors, and auto-populate DB.

Streamlit Client UI Layout

An intuitive front-end is developed using Streamlit to facilitate testing and administrative tasks:
1. Home Dashboard: Visualizes overall system stats (enrolled candidates, total embeddings) and health checks.
2. Candidate Enrollment: Allows live camera capture via browser media device, input metadata, and runs real-time checks.
3. Explorer View: Interactive tables showing registered candidates, metadata, option to delete or regenerate vector.
4. Data Management: Triggers bulk uploads/exports using JSON and CSV files.

6. Execution Guide

To run the application, use the following bash commands:

```
  # 1. Activate Virtual Environment
source .venv/bin/activate

# 2. Copy configuration environment variables
cp .env.example .env

# 3. Run database migration schema updates
alembic upgrade head

# 4. Launch FastAPI Server
bash scripts/run_server.sh

# 5. Launch Streamlit UI (runs in separate terminal on port 8501)
bash scripts/run_ui.sh
```